

```
;; The assembly-language translation of params.c
;; Obtained by "gcc -S params.c" into params.s
```

```
        .file      "params.c"
.globl globalN
        .data
        .align 4
        .type      globalN, @object
        .size      globalN, 4
globalN:
        .long      100
        .section    .rodata
        .align 8
.LC0:
        .long      0
        .long      -1073741824
        .align 8
.LC1:
        .long      0
        .long      1076101120
        .text
.globl ff1
        .type      ff1, @function
ff1:
        pushl      %ebp
        movl        %esp, %ebp
        subl        $8, %esp
        movl        8(%ebp), %eax
        movl        %eax, -8(%ebp)
        movl        12(%ebp), %eax
        movl        %eax, -4(%ebp)
        fldl        -8(%ebp)
        fmull       -8(%ebp)
        fldl        -8(%ebp)
        fldl        .LC0
        fmulp       %st, %st(1)
        faddp       %st, %st(1)
        fldl        .LC1
        faddp       %st, %st(1)
        leave
        ret
        .size      ff1, .-ff1
```

```

.section          .rodata
.LC3:
.string "1 : %d\n"
.LC4:
.string "2 : %d\n"
.LC5:
.string "3 : %d\n"
.LC6:
.string "4 : %d\n"
.LC7:
.string "5 : %d\n"
.LC8:
.string "GlobalN = %d\n"
.text
.globl main
.type    main, @function
main:
    leal    4(%esp), %ecx
    andl    $-16, %esp
    pushl   -4(%ecx)
    pushl   %ebp
    movl    %esp, %ebp
    pushl   %esi
    pushl   %ebx
    pushl   %ecx
    subl    $428, %esp
    movl    %ecx, -432(%ebp)
    movl    $9, -20(%ebp)                ; i = 9 in the for loop
    jmp     .L4
.L5:
    movl    $9, -16(%ebp)                ; j = 9 in the for loop
    jmp     .L6
.L7:
    movl    -432(%ebp), %eax
    subl    $1, (%eax)
    movl    -432(%ebp), %edx
    cmpl    $0, (%edx)
    je      .L8
    movl    -20(%ebp), %ebx
    movl    -16(%ebp), %esi
    movl    -432(%ebp), %edx
    movl    (%edx), %eax
    sall    $2, %eax
    movl    -432(%ebp), %edx
    addl    4(%edx), %eax
    movl    (%eax), %eax
    movl    %eax, (%esp)
    call    atoi

```

```

movl    %edx, %eax
movl    %ebx, %eax
sall    $2, %eax
addl    %ebx, %eax
addl    %eax, %eax
addl    %esi, %eax
movl    %edx, -420(%ebp,%eax,4)
jmp     .L10

.L8:
movl    -432(%ebp), %eax
movl    $1, (%eax)
movl    -20(%ebp), %ecx
movl    -16(%ebp), %ebx
movl    -20(%ebp), %edx
movl    %edx, %eax
sall    $2, %eax
addl    %edx, %eax
addl    %eax, %eax
movl    %eax, %edx
addl    -16(%ebp), %edx
movl    %ecx, %eax
sall    $2, %eax
addl    %ecx, %eax
addl    %eax, %eax
addl    %ebx, %eax
movl    %edx, -420(%ebp,%eax,4)

.L10:
subl    $1, -16(%ebp)

.L6:
cmpl    $0, -16(%ebp)
jne     .L7
subl    $1, -20(%ebp)

.L4:
cmpl    $0, -20(%ebp)
jne     .L5
leal    -420(%ebp), %eax
movl    $10, 4(%esp)
movl    %eax, (%esp)
call    fmat1
movl    %eax, 4(%esp)
movl    $.LC3, (%esp)
call    printf
leal    -420(%ebp), %eax
movl    $10, 4(%esp)
movl    %eax, (%esp)
call    fmat2
movl    %eax, 4(%esp)
movl    $.LC4, (%esp)

```

; Here the for loop ends
 ; Preparation for the
 ; call to fmat1
 ; using the result of fmat1
 ; Preparation for the
 ; call to fmat2
 ; using the result of fmat2

```

    call    printf
    leal    -420(%ebp), %eax        ; Preparation for the
    movl    $10, 4(%esp)           ; call to fmat3
    movl    %eax, (%esp)
    call    fmat3
    movl    %eax, 4(%esp)           ; using the result of fmat3
    movl    $.LC5, (%esp)
    call    printf
    movl    $10, 4(%esp)           ; Preparation for the
    leal    -420(%ebp), %eax        ; call to fmat4
    movl    %eax, (%esp)
    call    fmat4
    movl    %eax, 4(%esp)           ; using the result of fmat4
    movl    $.LC6, (%esp)
    call    printf
    leal    -420(%ebp), %eax        ; Preparation for the call
    movl    %eax, (%esp)           ; to fmat5 : note the
    call    fmat5                  ; difference from others
    movl    %eax, 4(%esp)           ; using the result of fmat5
    movl    $.LC7, (%esp)
    call    printf
    movl    globalN, %eax           ; the NAME globalN used here
    movl    %eax, 4(%esp)
    movl    $.LC8, (%esp)
    call    printf
    movl    $0, %eax
    addl    $428, %esp
    popl    %ecx
    popl    %ebx
    popl    %esi
    popl    %ebp
    leal    -4(%ecx), %esp
    ret
.size    main, .-main
.section    .rodata
.LC9:
    .string "%d "
    .text
.globl fmat1
.type    fmat1, @function
fmat1:
    pushl   %ebp                    ; operations at activation
    movl    %esp, %ebp
    subl    $24, %esp
    movl    $0, -4(%ebp)
    jmp     .L15
.L16:
    movl    -4(%ebp), %eax

```

```

    sall    $2, %eax
    addl    8(%ebp), %eax
    movl    (%eax), %eax
    movl    %eax, 4(%esp)
    movl    $.LC9, (%esp)
    call    printf
    addl    $1, -4(%ebp)
.L15:
    movl    -4(%ebp), %eax
    cmpl    12(%ebp), %eax
    jl      .L16
    movl    $10, (%esp)
    call    putchar
    movl    12(%ebp), %eax    ; the return value in eax
    leave
    ret
    .size   fmat1, .-fmat1
    .data
    .align  4
    .type   k.1858, @object
    .size   k.1858, 4
k.1858:
    .long   -100
    .section        .rodata
.LC10:
    .string "%d "
    .text
.globl fmat2
    .type   fmat2, @function
fmat2:
    pushl   %ebp                ; operations at activation
    movl    %esp, %ebp          ; what happened to static k?
    subl    $24, %esp
    movl    $0, -8(%ebp)
    jmp     .L20
.L21:
    movl    $0, -4(%ebp)
    jmp     .L22
.L23:
    movl    -8(%ebp), %eax
    sall    $2, %eax
    addl    8(%ebp), %eax
    movl    (%eax), %edx
    movl    -4(%ebp), %eax
    sall    $2, %eax
    leal    (%edx,%eax), %eax
    movl    (%eax), %eax
    movl    %eax, 4(%esp)

```

```

        movl    $.LC10, (%esp)
        call    printf
        addl    $1, -4(%ebp)
.L22:
        movl    -4(%ebp), %eax
        cmpl    12(%ebp), %eax
        jl      .L23
        addl    $1, -8(%ebp)
.L20:
        movl    -8(%ebp), %eax
        cmpl    12(%ebp), %eax
        jl      .L21
        movl    -8(%ebp), %eax
        movl    %eax, %edx
        imull    -4(%ebp), %edx
        movl    k.1858, %eax                ; this prepares the return
        leal    (%edx,%eax), %eax          ; value using the static k.
        movl    %eax, k.1858
        movl    $10, (%esp)
        call    putchar
        movl    k.1858, %eax
        leave
        ret
        .size    fmat2, .-fmat2
        .section      .rodata
.LC11:
        .string  "%d "
        .text
.globl fmat3
        .type    fmat3, @function
fmat3:
        pushl    %ebp
        movl    %esp, %ebp
        subl    $24, %esp
        movl    $0, -8(%ebp)
        jmp     .L28
.L29:
        movl    $0, -4(%ebp)
        jmp     .L30
.L31:
        movl    -8(%ebp), %eax
        sall    $2, %eax
        addl    8(%ebp), %eax
        movl    (%eax), %edx
        movl    -4(%ebp), %eax
        sall    $2, %eax
        leal    (%edx,%eax), %eax
        movl    (%eax), %eax

```

```

        movl    %eax, 4(%esp)
        movl    $.LC11, (%esp)
        call    printf
        addl    $1, -4(%ebp)
.L30:
        cmpl    $9, -4(%ebp)
        jle     .L31
        addl    $1, -8(%ebp)
.L28:
        movl    -8(%ebp), %eax
        cmpl    12(%ebp), %eax
        jl      .L29
        movl    $10, (%esp)
        call    putchar
        movl    12(%ebp), %edx
        movl    %edx, %eax
        sall    $2, %eax
        addl    %edx, %eax
        addl    %eax, %eax
        leave
        ret
        .size   fmat3, .-fmat3
        .section        .rodata
.LC12:
        .string "$%d "
        .text
.globl fmat4
        .type    fmat4, @function
fmat4:
        pushl    %ebp
        movl    %esp, %ebp
        subl    $24, %esp
        movl    $0, -8(%ebp)
        jmp     .L36
.L37:
        movl    $0, -4(%ebp)
        jmp     .L38
.L39:
        movl    -8(%ebp), %edx
        movl    %edx, %eax
        sall    $2, %eax
        addl    %edx, %eax
        sall    $3, %eax
        movl    %eax, %edx
        addl    8(%ebp), %edx
        movl    -4(%ebp), %eax
        movl    (%edx,%eax,4), %eax
        movl    %eax, 4(%esp)

```

```

        movl    $.LC12, (%esp)
        call    printf
        addl    $1, -4(%ebp)
.L38:
        cmpl    $9, -4(%ebp)
        jle     .L39
        addl    $1, -8(%ebp)
.L36:
        movl    -8(%ebp), %eax
        cmpl    12(%ebp), %eax
        jl      .L37
        movl    $10, (%esp)
        call    putchar
        movl    12(%ebp), %edx
        movl    %edx, %eax
        sall    $2, %eax
        addl    %edx, %eax
        addl    %eax, %eax
        leave
        ret
        .size   fmat4, .-fmat4
        .section        .rodata
.LC13:
        .string "%d "
        .text
.globl fmat5
        .type   fmat5, @function
fmat5:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $24, %esp
        movl    $0, -8(%ebp)
        jmp     .L44
.L45:
        movl    $0, -4(%ebp)
        jmp     .L46
.L47:
        movl    -8(%ebp), %edx
        movl    %edx, %eax
        sall    $2, %eax
        addl    %edx, %eax
        sall    $3, %eax
        movl    %eax, %edx
        addl    8(%ebp), %edx
        movl    -4(%ebp), %eax
        movl    (%edx,%eax,4), %eax
        movl    %eax, 4(%esp)
        movl    $.LC13, (%esp)

```

```

        call    printf
        addl    $1, -4(%ebp)
.L46:
        cmpl    $9, -4(%ebp)
        jle     .L47
        addl    $1, -8(%ebp)
.L44:
        cmpl    $9, -8(%ebp)
        jle     .L45
        movl    $10, (%esp)
        call    putchar
        movl    $100, %eax
        leave
        ret
        .size   fmat5, .-fmat5
        .ident   "GCC: (GNU) 4.1.0 (SUSE Linux)"
        .section .note.GNU-stack,"",@progbits

```